

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 1

Case Study

COURSE NAME: DEEP LEARNING

COURSE CODE:MLA0408

COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25%

Assessment Tool Weightage: 25%

QUESTIONS:

Total

Marks: 25

Case Study 1: CNN for Automated Medical Image Classification

A healthcare organization wants to develop a deep learning system to automatically classify chest X-ray images into normal and disease-affected categories. The dataset contains thousands of X-ray images with variations in size, brightness and image quality. A CNN model is proposed because it can automatically learn important visual features such as edges, textures, and abnormal regions without manually designing features.

Question:

- Design and explain a suitable CNN architecture for this medical image classification system.
- Describe the motivation for using CNNs and explain the role of convolutional layers, filters, pooling layers, and fully connected layers in the proposed architecture.
- Explain how parameter sharing reduces the number of trainable parameters compared with a fully connected network.
- Discuss suitable regularization techniques such as dropout, data augmentation and batch normalization to reduce overfitting.
- Finally, explain whether AlexNet or ResNet would be more appropriate for this application and justify your choice based on feature learning, network depth, computational requirements and classification performance.

Case Study 2: CNN-Based Autonomous Vehicle Object Recognition

An autonomous vehicle company is developing a vision system that must identify cars, pedestrians, traffic signs, bicycles, and road obstacles from images captured by cameras mounted on the vehicle. The system must recognize objects under different lighting conditions, viewing angles, distances and road

environments. The development team is considering AlexNet and ResNet architectures for building the CNN-based recognition system.

Question:

- Analyze how a CNN architecture can be designed for this autonomous vehicle application.
- Explain the purpose of the input, convolution, activation, pooling, and fully connected/output layers and describe how convolutional filters progressively learn low-level and high-level features.
- Explain the importance of parameter sharing in reducing computational complexity and improving translation-related feature detection.
- Discuss the possible problem of overfitting and recommend suitable regularization methods.
- Compare AlexNet and ResNet in terms of architecture, depth, parameter efficiency, training difficulty, vanishing-gradient problems and feature-learning capability.
- Finally, recommend the most suitable architecture for the autonomous vehicle system and justify your decision based on accuracy, computational cost and real-time application requirements.

Code of Conduct:

I **G. Naveen Kumar Reddy (192525288)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **G. N.K.Reddy**

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided

Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand
---------	----	--	---	-------------------------------	---------------------------------------

ANSWERS

The following solutions are structured to satisfy all five rubric criteria: Understanding of Case, Application of Concepts, Analysis, Solution Design, and Clarity.

Case Study 1: CNN for Automated Medical Image Classification — Answers

1. Proposed CNN Architecture

Since chest X-ray images contain fine-grained, localized abnormalities (opacities, nodules, consolidations) against a fairly uniform anatomical background, a moderately deep CNN with batch normalization and global average pooling is proposed:

- **Input Layer:** $224 \times 224 \times 1$ (grayscale) or $224 \times 224 \times 3$ X-ray images, resized and pixel-normalized to $[0,1]$.
- **Preprocessing:** Contrast Limited Adaptive Histogram Equalization (CLAHE) to correct brightness/contrast variation across scanners.
- **Convolution Block 1:** Conv(32 filters, 3×3 , ReLU) $\rightarrow$ Batch Normalization $\rightarrow$ Conv(32, 3×3 , ReLU) $\rightarrow$ Max Pooling (2×2).
- **Convolution Block 2:** Conv(64, 3×3 , ReLU) $\rightarrow$ Batch Normalization $\rightarrow$ Conv(64, 3×3 , ReLU) $\rightarrow$ Max Pooling (2×2).
- **Convolution Block 3:** Conv(128, 3×3 , ReLU) $\rightarrow$ Batch Normalization $\rightarrow$ Conv(128, 3×3 , ReLU) $\rightarrow$ Max Pooling (2×2).
- **Convolution Block 4:** Conv(256, 3×3 , ReLU) $\rightarrow$ Batch Normalization $\rightarrow$ Max Pooling (2×2).
- **Global Average Pooling:** Replaces a large flatten layer, reduces parameters and overfitting.
- **Fully Connected Layer:** Dense(256, ReLU) $\rightarrow$ Dropout(0.5).
- **Output Layer:** Dense(1, Sigmoid) for binary classification (Normal vs Disease-affected); Softmax if multiple disease classes are required.
- **Training setup:** Adam optimizer, binary cross-entropy loss, early stopping on validation AUC, class-weighting to handle imbalance between normal and disease cases.

In practice, this custom network is best combined with transfer learning — initializing the convolutional base with ImageNet-pretrained ResNet weights and fine-tuning on the X-ray dataset — since medical datasets are usually limited in size (the approach used by systems such as CheXNet).

2. Motivation for CNNs and Role of Each Layer

Motivation

Traditional machine learning requires manually engineered features (edge detectors, texture descriptors) which are difficult to design for subtle radiological abnormalities. CNNs remove this bottleneck by learning features directly from pixel data through backpropagation. Their local-connectivity and weight-sharing properties make them naturally suited to images, where meaningful patterns (fluid buildup, opacities) are

spatially localized, and where the same abnormality can appear anywhere in the image — a property convolution inherently exploits.

Role of Each Component

- **Convolutional layers:** Slide small filters across the image to compute feature maps, capturing edges and gradients in early layers and progressively more complex structures (rib boundaries, lung opacity patterns) in deeper layers.
- **Filters (kernels):** Small learnable weight matrices (e.g., 3×3) that each specialize in detecting one type of local pattern; a layer typically has many filters producing many feature maps.
- **Pooling layers:** Down-sample feature maps (e.g., 2×2 max pooling), reducing spatial resolution and computation while providing a degree of translation invariance, so a small shift in the abnormality's position does not change the prediction.
- **Fully connected layers:** Take the high-level, flattened feature representation and combine it to produce the final decision boundary between 'normal' and 'disease-affected' classes.

3. Parameter Sharing

In a fully connected (dense) layer, every input pixel is connected to every neuron in the next layer with its own independent weight. For a $224 \times 224 \times 3$ image connected to just 128 neurons, the number of weights is:

$224 \times 224 \times 3 \times 128 \approx 19.2$ million parameters — for a single layer.

A convolutional layer instead uses one small filter (e.g., $3 \times 3 \times 3$) that is reused ("shared") at every spatial location of the image. With 32 such filters, the parameter count is:

$(3 \times 3 \times 3 + 1) \times 32 = 864$ parameters — regardless of image size.

This parameter sharing means the network learns one generic edge/texture detector and applies it everywhere in the image, rather than learning a separate detector for every pixel location. The benefits are:

- A drastic reduction (often by several orders of magnitude) in trainable parameters.
- Lower risk of overfitting, which is critical given limited labeled medical image data.
- Translation invariance — an abnormality is detected the same way no matter where it appears in the X-ray.
- Feasibility of building much deeper networks within practical memory and compute budgets.

4. Regularization Techniques to Reduce Overfitting

- **Dropout:** Randomly deactivates a fraction of neurons (e.g., 50%) in the fully connected layers during training, preventing the network from relying on any single feature and forcing redundant, robust representations.
- **Data Augmentation:** Since labeled medical images are limited, augmentations such as random rotation (± 10 – 15°), horizontal flipping (where anatomically valid), zoom, brightness/contrast jitter, and slight elastic deformation artificially expand the effective dataset and expose the model to the imaging variability (different machines, exposure settings) described in the case.
- **Batch Normalization:** Normalizes the activations of each layer to zero mean/unit variance per mini-batch, which stabilizes and accelerates training, allows higher learning rates, and provides a mild regularizing effect that reduces dependence on precise weight initialization.
- **Additional measures:** L2 weight decay, early stopping based on validation loss/AUC, and transfer learning from a pretrained backbone all further reduce the risk of the model memorizing the training set.

5. AlexNet vs. ResNet — Recommendation

AlexNet (2012) has 8 layers with large early filters (11×11 , 5×5), ReLU activations, local response normalization and dropout. It performs adequately on moderately sized datasets but its accuracy plateaus because it cannot be made much deeper — stacking additional plain convolutional layers causes the vanishing-gradient problem, where gradients shrink as they are back-propagated through many layers, so early layers stop learning effectively.

ResNet introduces residual (skip) connections that add a layer's input directly to its output (identity mapping). This allows gradients to flow directly backward through the shortcut paths, so networks of 50, 101, or even 152 layers can be trained without degradation. Greater depth means the network can learn a much richer hierarchy of features — critical for detecting subtle, low-contrast abnormalities such as early-stage opacities in chest X-rays.

Recommendation: ResNet is more appropriate for this application.

- Feature learning: Its deeper residual blocks extract far more discriminative, fine-grained features needed to distinguish subtle disease patterns from normal anatomy.
- Network depth: Skip connections allow ResNet to go deep (e.g., ResNet-50) without the vanishing-gradient degradation that limits AlexNet.
- Computational requirements: ResNet has more layers but is comparatively parameter-efficient (using 3×3 filters and global average pooling instead of large fully connected layers); this cost is acceptable in a healthcare setting where diagnostic accuracy is prioritized over real-time inference and adequate GPU infrastructure is typically available.
- Classification performance: Pretrained ResNet backbones (via transfer learning) consistently outperform AlexNet on medical imaging benchmarks and are the basis of real-world systems such as CheXNet-style pneumonia detectors.

Case Study 2: CNN-Based Autonomous Vehicle Object Recognition — Answers

1. CNN Architecture Design for the Autonomous Vehicle Application

Unlike the single-label medical classification task, this application requires detecting and localizing multiple object categories (cars, pedestrians, traffic signs, bicycles, obstacles) simultaneously within a scene, under varying lighting, distance and viewing angle. This calls for an object-detection CNN rather than a simple classifier:

- **Input:** RGB camera frames (e.g., resized to 416×416 or 608×608), normalized, captured across day/night and weather conditions.
- **Backbone (feature extractor):** A deep CNN such as ResNet-50 (or a lighter variant for embedded hardware) extracts hierarchical feature maps at multiple depths.
- **Multi-scale feature aggregation:** A Feature Pyramid Network (FPN) combines shallow (fine detail, small/far objects) and deep (semantic, large/near objects) feature maps so that both distant pedestrians and nearby vehicles are detected reliably.
- **Detection head:** Convolutional prediction layers (as in Faster R-CNN, SSD, or YOLO-style architectures) output bounding-box coordinates and class probabilities for each anchor location.
- **Output:** A set of bounding boxes with class labels (car, pedestrian, traffic sign, bicycle, obstacle) and confidence scores, refined using non-maximum suppression.
- **Robustness measures:** Training on diverse driving datasets (e.g., KITTI, BDD100K, Cityscapes) covering different lighting and weather, combined with heavy augmentation, ensures the network generalizes across the conditions described in the case.

2. Purpose of Each Layer and Progressive Feature Learning

- **Input layer:** Receives raw pixel intensities (RGB channels) from the vehicle's cameras.
- **Convolution layers:** Apply learnable filters to detect spatial patterns; stacked convolutions build a feature hierarchy.
- **Activation (ReLU):** Introduces non-linearity after each convolution, enabling the network to model complex, non-linear relationships between pixels and object categories; also mitigates vanishing gradients better than sigmoid/tanh.
- **Pooling layers:** Downsample feature maps (max pooling), reducing computation and providing invariance to small translations/scale changes — useful when the same object appears at different positions or distances.
- **Fully connected / output layers:** In classification-style heads, dense layers combine global features for a final decision; in detection heads, 1×1 convolutions predict per-location class scores and bounding-box offsets.

Progressive feature learning: the earliest convolutional layers learn generic low-level features such as edges, corners and colour gradients. Middle layers combine these into textures and object parts (wheels, headlights, limb shapes, sign borders). The deepest layers assemble parts into high-level, semantically meaningful representations of whole objects — a car's silhouette, a pedestrian's pose, or a traffic sign's shape — which the detection head uses to classify and localize each object.

3. Importance of Parameter Sharing

Because a convolutional filter's weights are shared across every spatial location of the input, the same detector (e.g., a wheel or edge detector) is applied uniformly across the entire frame. This has two direct benefits for this application:

- **Reduced computational complexity:** The number of learnable parameters depends only on filter size and channel depth, not on image resolution, keeping the model compact enough for near-real-time inference on the vehicle's onboard hardware, unlike a fully connected network whose parameter count would explode with image size.
- **Translation-related feature detection:** A pedestrian or vehicle must be recognized correctly no matter where it appears in the frame — at the centre, edge, near or far. Parameter sharing gives the network this translation invariance inherently, so a single trained detector generalizes across all spatial positions instead of needing separate weights per location.

4. Overfitting and Recommended Regularization

A key risk is that the model may overfit to the specific cameras, geographic regions, or lighting/weather conditions present in the training data, leading to poor generalization on unseen roads, unfamiliar sign designs, or new weather (fog, rain, night driving) — a safety-critical failure mode for an autonomous vehicle.

- **Data augmentation:** Simulate brightness/contrast changes, shadows, rain/fog effects, random cropping, scaling and horizontal flipping to expose the model to the variety of lighting, angles and distances described in the case.
- **Dropout:** Applied in the fully connected/detection head layers to prevent reliance on narrow feature combinations.
- **Batch normalization:** Stabilizes training across diverse batches of varied lighting/scene conditions and mildly regularizes the network.
- **L2 weight decay & early stopping:** Penalize excessively large weights and halt training once validation performance on a held-out, diverse driving set stops improving.
- **Transfer learning & diverse datasets:** Pretraining on large-scale datasets (ImageNet) and fine-tuning on multiple driving datasets (KITTI, BDD100K, Cityscapes) covering different countries, weather and times of day improves generalization beyond any single data source.

5. AlexNet vs. ResNet — Comparison

Aspect	AlexNet	ResNet
Architecture	5 convolutional layers + 3 fully connected layers; large early filters (11×11, 5×5)	Stacked residual blocks with small 3×3 filters and identity skip connections; batch normalization throughout
Depth	8 weight layers	18 to 152+ weight layers (e.g., ResNet-50, ResNet-101)
Parameter efficiency	~60 million parameters, mostly concentrated in the large fully connected layers	Fewer parameters relative to depth; uses global average pooling instead of large dense layers
Training difficulty	Relatively easy and fast to train, but cannot be extended much deeper	Deeper and more complex, but residual connections make optimization tractable even at great depth

Vanishing gradient	Susceptible if depth is increased; gradients shrink through many plain stacked layers	Solved via identity shortcut connections that let gradients flow directly to earlier layers
Feature-learning capability	Learns useful but comparatively shallow feature hierarchies	Learns much richer, multi-scale hierarchical features, better suited to complex, cluttered scenes

6. Recommended Architecture and Justification

Recommendation: ResNet (e.g., ResNet-50 as backbone, or a lighter ResNet-18/34 variant for embedded deployment).

- **Accuracy:** Detecting cars, pedestrians, cyclists, signs and obstacles reliably under varied lighting, angle and distance is safety-critical; ResNet's depth and residual learning yield significantly higher detection accuracy than AlexNet, especially for small or partially occluded objects such as distant pedestrians.
- **Computational cost:** Although ResNet is more computationally demanding than AlexNet, its parameter efficiency (small filters, global average pooling) combined with model compression techniques (pruning, quantization) and lighter variants (ResNet-18/34) or efficient detection heads (e.g., YOLO with a ResNet-style backbone) make it deployable on automotive-grade GPUs/edge accelerators.
- **Real-time application requirements:** A properly optimized ResNet-based detector (using techniques such as TensorRT acceleration, quantization, or a reduced-depth backbone) can meet the frame-rate requirements of real-time driving decisions, while still providing the depth needed for robust feature extraction — a balance AlexNet cannot match at comparable accuracy.

Overall, ResNet's superior accuracy and robust feature learning outweigh its higher computational cost, and that cost can be managed through architectural optimization — making it the more suitable choice for a safety-critical autonomous vehicle perception system.